



RaceDay

API Endpoint Plan

South African Road Running, Walking and Cycling Event Management System

Prepared from the RaceDay project brief, supplied database/ERD structure, and the API endpoint teaching guide. The plan covers Authentication, User Profile, Events, Categories, Event Enrolments and Results, with additional Routes and Weather endpoints represented in the RaceDay model and project description.

RaceDay convention	Value
API base	/api/
Primary roles	Admin, Organiser, Participant
Response format	JSON
Endpoint style	Resource-oriented REST routes using HTTP methods and consistent naming

Document purpose. This document defines the API contract to be followed when implementing Part 2. Each endpoint specifies its HTTP method, route, purpose, required role, request body and expected response. The plan is aligned to the supplied RaceDay schema: Role, User, Event, Weather, Route, Category, Registration and Result.



API. Design Conventions

Resource-oriented URLs, HTTP methods, consistent naming and protected ownership rules.

The supplied teaching guide recommends resource-oriented URLs, nouns rather than verbs, plural resource names, HTTP methods for CRUD operations, consistent URL patterns, and versioning when breaking changes must be managed. For this RaceDay assignment, the route convention remains under `/api/` to keep the implementation contract explicit and consistent.

Method	Purpose	RaceDay use
GET	Read	Retrieve events, profiles, categories, routes, weather and results.
POST	Create	Create accounts, events, categories, routes, weather records, registrations and results.
PATCH	Partial update	Update selected profile, event, category, route, weather, registration or result fields.
DELETE	Remove	Remove or cancel resources where the relevant business rules allow it.

Request data conventions. Path parameters identify a specific resource, such as `/api/events/{id}`. Query parameters are used for optional filtering, sorting or pagination. Headers carry authentication and content metadata. Request bodies carry JSON data for POST and PATCH operations.

Role enforcement. Protected endpoints should derive the current user from the authenticated request rather than trusting a client-supplied user ID for ownership. Participant-only resources must be restricted to the participant's own records, while organiser operations must be restricted to events they are permitted to manage.

Status-code convention. 200 OK successful retrieval/update; 201 Created successful creation; 204 No Content successful removal; 400 Bad Request invalid input; 401 Unauthorized missing/invalid authentication; 403 Forbidden authenticated but not permitted; 404 Not Found resource does not exist; 409 Conflict request conflicts with an existing record or business rule.



1. Authentication

Account creation and login establish the identity used by protected RaceDay endpoints.

HTTP Method	Route	Description	Role Required	Request Body (if any)	Expected Response
POST	/api/auth/register	Create a new RaceDay user account. New public accounts are created as Participant unless the system explicitly assigns another permitted role.	Public	<code>{"fullName","email","password","phoneNumber"}</code>	201 Created — user created; 400 Bad Request for invalid/duplicate data.
POST	/api/auth/login	Authenticate a registered user and start an authenticated session.	Public	<code>{"email","password"}</code>	200 OK — authenticated user details and access token; 401 Unauthorized for invalid credentials.
POST	/api/auth/logout	End the current authenticated session.	Authenticated	None	204 No Content — session ended; 401 Unauthorized if authentication is invalid.



2. User Profile

The User table stores identity, contact details, role and account status.

HTTP Method	Route	Description	Role Required	Request Body (if any)	Expected Response
GET	/api/users/me	Return the profile of the currently authenticated user.	Authenticated	None	200 OK — user profile; 401 Unauthorized if not authenticated.
PATCH	/api/users/me	Update editable details for the currently authenticated user's profile.	Authenticated	{"fullName"?, "phoneNumber"?, "email"?"}	200 OK — updated profile; 400 Bad Request for invalid data; 409 Conflict for an email already in use.
GET	/api/users/{id}	View a user's public profile where permitted, primarily for event administration and participant identification.	Organiser / Admin	None	200 OK — user profile; 404 Not Found if the user does not exist; 403 Forbidden if access is not allowed.



3. Events

Events are created and managed by organisers and browsed by participants.

HTTP Method	Route	Description	Role Required	Request Body (if any)	Expected Response
GET	/api/events	List RaceDay events so participants can browse upcoming and available events.	Public	None	200 OK — event collection; optional query filters may include date, location, or status.
GET	/api/events/{id}	Return full details for one event, including its organiser, date, location, status, route/category information where available.	Public	None	200 OK — event details; 404 Not Found if the event does not exist.
POST	/api/events	Create a new running, walking, or cycling event.	Organiser / Admin	{"eventName", "description", "eventDate", "location", "status"?"}	201 Created — event record; 400 Bad Request for invalid data.
PATCH	/api/events/{id}	Update editable event details such as name, description, date, location, or status.	Organiser / Admin	{"eventName"?, "description"?, "eventDate"?, "location"?, "status"?"}	200 OK — updated event; 404 Not Found if absent; 403 Forbidden if the organiser does not own it.
DELETE	/api/events/{id}	Remove or cancel an event according to the system's event-management rules.	Organiser / Admin	None	204 No Content — event removed/cancelled; 404 Not Found if absent; 403 Forbidden if not permitted.



4. Event Categories

Categories connect an event to a route and define entry rules such as distance, fee, capacity and age/gender restrictions.

HTTP Method	Route	Description	Role Required	Request Body (if any)	Expected Response
GET	/api/events/{eventId}/categories	List the entry categories available for a specific event.	Public	None	200 OK — category collection; 404 Not Found if the event does not exist.
GET	/api/categories/{id}	Return details for one event category, including distance, fee, participant limit, gender restriction, and age group.	Public	None	200 OK — category details; 404 Not Found if absent.
POST	/api/events/{eventId}/categories	Create an entry category for an event and link it to a route.	Organiser / Admin	{"routeId", "categoryName", "distanceKM", "entryFee", "maxParticipants", "genderRestriction", "ageGroup"}	201 Created — category record; 400 Bad Request for invalid data; 404 Not Found for an invalid event/route.

HTTP Method	Route	Description	Role Required	Request Body (if any)	Expected Response
PATCH	/api/categories/{id}	Update editable category information such as fee, capacity, restrictions, or route.	Organiser / Admin	{"routeId"?:,"categoryName"?:,"distanceKM"?:,"entryFee"?:,"maxParticipants"?:,"genderRestriction"?:,"ageGroup"?:}	200 OK — updated category; 404 Not Found if absent.
DELETE	/api/categories/{id}	Remove an event category when it is no longer offered and removal is allowed by the business rules.	Organiser / Admin	None	204 No Content — category removed; 404 Not Found if absent; 409 Conflict if registrations prevent deletion.



5. Event Enrolments / Registrations

Registration links a participant to a category and stores the registration date, bib number and status.

HTTP Method	Route	Description	Role Required	Request Body (if any)	Expected Response
POST	/api/events/{eventId}/categories/{categoryId}/enrolments	Register the authenticated participant for a selected event category and create a registration record.	Participant	{"bibNumber"}	201 Created — registration created with RegistrationDate and RegistrationStatus; 404 Not Found for missing event/category; 409 Conflict for invalid/duplicate enrolment or unavailable capacity.
GET	/api/users/me/enrolments	List all event registrations belonging to the authenticated participant.	Participant	None	200 OK — participant's registration collection; 401 Unauthorized if not authenticated.
GET	/api/events/{eventId}/enrolments	List registrations for an event so an organiser can manage participants.	Organiser / Admin	None	200 OK — registration collection; 404 Not Found if event does not exist; 403 Forbidden if access is not allowed.
GET	/api/enrolments/{id}	Return the details of a specific registration.	Participant (own) / Organiser / Admin	None	200 OK — registration details; 403 Forbidden for another participant's registration; 404 Not Found if absent.
PATCH	/api/enrolments/{id}	Update the registration status where permitted, for example confirming or cancelling a registration.	Organiser / Admin	{"registrationStatus"}	200 OK — updated registration; 400 Bad Request for an invalid status; 404 Not Found if absent.
DELETE	/api/enrolments/{id}	Cancel the authenticated participant's own registration.	Participant (own) / Organiser / Admin	None	204 No Content — registration cancelled/removed; 403 Forbidden for another participant's record; 404 Not Found if absent.



6. Results

Results are linked to registrations and store start/finish times, position, pace and result status.

HTTP Method	Route	Description	Role Required	Request Body (if any)	Expected Response
GET	/api/events/{eventId}/results	List published results for an event, ordered by position where available.	Public	None	200 OK — result collection; 404 Not Found if event does not exist.
GET	/api/users/me/results	Show the authenticated participant's personal performance history.	Participant	None	200 OK — result history; 401 Unauthorized if not authenticated.
GET	/api/registrations/{registrationId}/result	Return the result linked to a specific registration.	Participant (own) / Organiser / Admin	None	200 OK — result details; 403 Forbidden for an unauthorised participant; 404 Not Found if no result exists.

HTTP Method	Route	Description	Role Required	Request Body (if any)	Expected Response
POST	/api/registrations/{registrationId}/result	Record a participant's race result after an event.	Organiser / Admin	{"startTime","finishTime"?,"position"?,"pace"?,"resultStatus"?}	201 Created — result record; 400 Bad Request for invalid timing/position/pace; 404 Not Found if registration is absent.
PATCH	/api/results/{id}	Correct or update an existing race result.	Organiser / Admin	{"startTime"?,"finishTime"?,"position"?,"pace"?,"resultStatus"?}	200 OK — updated result; 404 Not Found if absent.



7. Routes

Routes provide distance, elevation, start/end points and map information used by participants to prepare for an event.

HTTP Method	Route	Description	Role Required	Request Body (if any)	Expected Response
GET	/api/events/{eventId}/routes	List route information associated with an event.	Public	None	200 OK — route collection; 404 Not Found if event does not exist.
GET	/api/routes/{id}	Return route details including distance, elevation gain, start/end points, and map URL.	Public	None	200 OK — route details; 404 Not Found if absent.
POST	/api/events/{eventId}/routes	Create a route for an event.	Organiser / Admin	{"routeName","distanceKM","elevationGain","startPoint","endPoint","mapDataURL"?}	201 Created — route record; 400 Bad Request for invalid distance/elevation; 404 Not Found if event is absent.
PATCH	/api/routes/{id}	Update route information for an event.	Organiser / Admin	{"routeName"?,"distanceKM"?,"elevationGain"?,"startPoint"?,"endPoint"?,"mapDataURL"?}	200 OK — updated route; 404 Not Found if absent.
DELETE	/api/routes/{id}	Remove a route when it is not required by an active category.	Organiser / Admin	None	204 No Content — route removed; 404 Not Found if absent; 409 Conflict if dependent categories prevent deletion.



8. Weather

Weather information supports the project requirement for participants to prepare for race day using weather and forecast conditions.

HTTP Method	Route	Description	Role Required	Request Body (if any)	Expected Response
GET	/api/events/{eventId}/weather	Provide weather information associated with an event so participants can prepare for race day.	Public	None	200 OK — weather data including temperature, condition, wind speed, humidity, and forecast time; 404 Not Found if unavailable.
POST	/api/events/{eventId}/weather	Add or record weather/forecast information for an event.	Organiser / Admin	{"temperature","weatherCondition","windSpeed","humidity","forecastTime"}	201 Created — weather record; 400 Bad Request for invalid humidity/wind values; 404 Not Found if event is absent.
PATCH	/api/weather/{id}	Update an event's stored weather/forecast information.	Organiser / Admin	{"temperature"?,"weatherCondition"?,"windSpeed"?,"humidity"?,"forecastTime"?}	200 OK — updated weather record; 404 Not Found if absent.



9. Administrative User Management

Additional endpoints expose administration implied by the supplied Role and User structures.

HTTP Method	Route	Description	Role Required	Request Body (if any)	Expected Response
GET	/api/users	List registered users for administration and event-management purposes.	Admin	None	200 OK — user collection; 403 Forbidden if the caller is not an administrator.
PATCH	/api/users/{id}/role	Change a user's role where the system permits role administration.	Admin	{"roleId"}	200 OK — updated role; 400 Bad Request for an invalid role; 404 Not Found if user/role is absent.
PATCH	/api/users/{id}/status	Activate or deactivate a user account.	Admin	{"accountStatus"}	200 OK — updated account status; 400 Bad Request for an invalid status; 404 Not Found if user is absent.



. Implementation Alignment

The endpoint plan acts as the API contract for Part 2 and follows the supplied RaceDay relationships.

Implementation alignment. Implemented routes, HTTP methods, roles, request fields and response behaviour should remain consistent with this document unless a justified change is documented.

Required project area	Planned coverage
Authentication	Register, login and logout
User Profile	Current-user profile, permitted user lookup and administration
Events	Browse, view, create, update and remove/cancel events
Categories	Browse, create, update and remove event entry categories
Event Enrolments	Participant registration, own registrations, organiser management and cancellation
Results	Event results, personal history, create and correct results
Additional RaceDay resources	Routes and Weather, represented in the supplied data model and project description

RaceDay resource-to-database alignment. Role → User → Event → Weather / Route → Category → Registration → Result. The endpoint structure follows the supplied relationships. A Registration references User and Category, while Result references Registration.

Implementation principle. Build a working vertical slice around authentication, events and registrations, then layer on categories, results, routes and weather. Validate ownership and role permissions on the server and return clear HTTP status codes and JSON responses for successful and failed requests.